

Lecture 3

File system & Essential Shell Commands

CoSc2043 – Network and System Administration

Topics

1. Terminal
2. Shell
3. Commands
4. File
5. File Permission

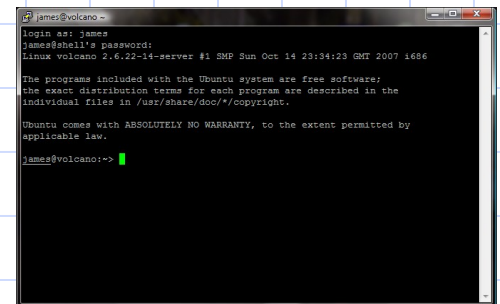
A Quick Start

◆ Terminal

◆ PuTTY, a telnet/ssh client

- a free and open source terminal emulator application
 - ◆ a window in your desktop that works like old time terminal

◆ Enter your ID and password



Your first encounter: shell

- **Shell**: a special-purpose program, **command line interpreter**, read commands typed by a user and execute programs in response to entered commands
- Many different shells:
 - **Bourne Shell (sh)**: oldest,
 - I/O redirection, pipelines, filename generation (globbing), variables, environment variables, command substitution, background command execution, function
 - **C Shell (csh)**: syntax of flow-control similar to C, command history, command-line editing, job control, aliases
 - **Korn Shell (ksh)**: “csh”, compatible with sh
 - **Bourne again Shell (bash)**: GNU’s reimplementation of Bourne shell, supports features added in C shell, and Korn shell

Check/Change Login Shell

- ◆ To check the shell you are using
 - `echo $SHELL`
 - `echo $0`
- ◆ login shell: default shell for a user, specified in `/etc/passwd`
- ◆ To change your login shell, use command
 - `chsh`

Shell: interactive mode

- A shell session (a dialog between user and shell)
 1. Displays a **prompt** character, and waits for user to type in a **command line**
 - ♦ Prompt depends on shell: sh, ksh, bash: \$ csh: % tcsh: >
 - ♦ May be customized (with current directory, host, ...)
 2. On input of a **command line**, shell extracts **command name and arguments**, searches for the program, and runs it.
 3. When program finishes, shell continues to step 1
 4. The loop continues until user types "exit" or "ctrl-d" to end

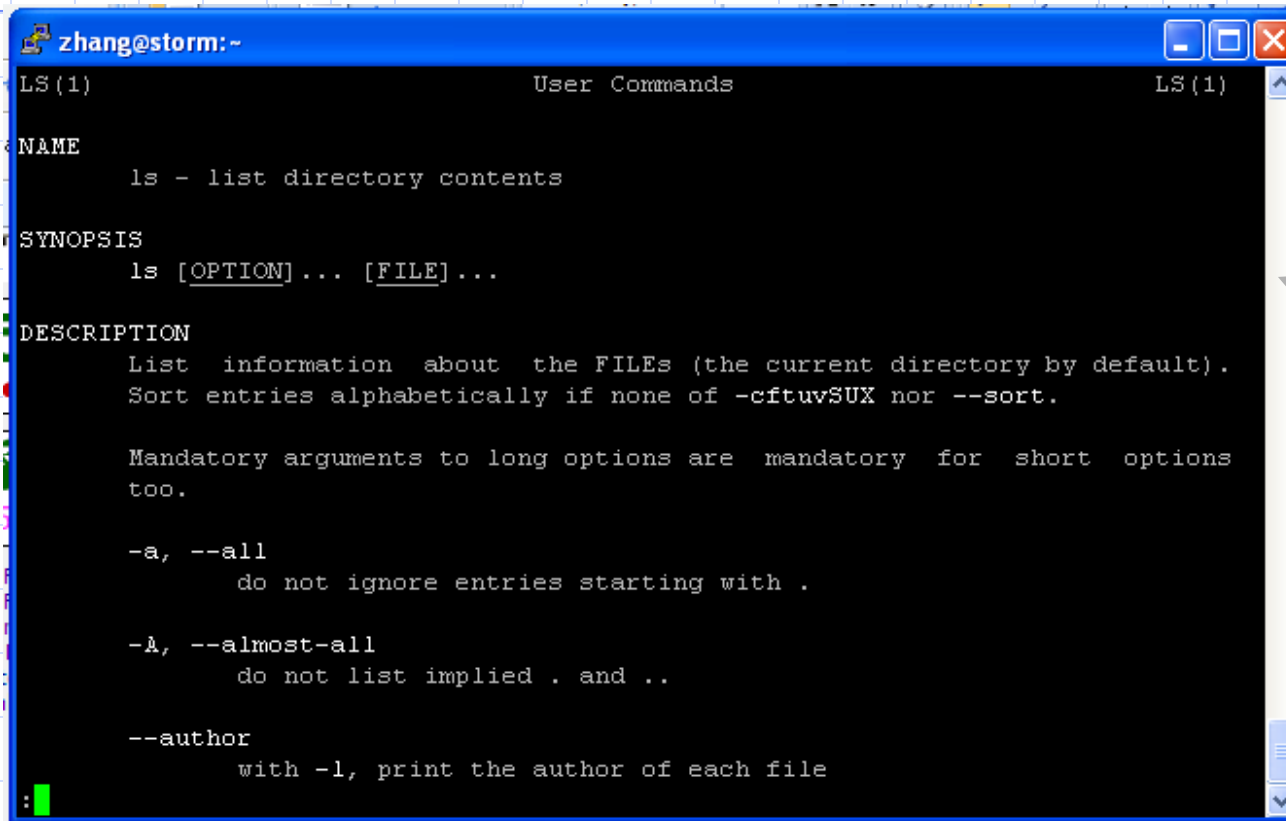
UNIX command line

- Command name and arguments:

command [[-] option (s)] [option argument (s)] [command argument (s)]

- **Command arguments** are mostly file or directory names
 - `cp prog1.cpp prog1.cpp.bak`
- **Options**: used to control behavior of the command
 - `head -20 lab1.cpp`
 - `wc -w lab2.cpp` // count how many words
 - Some options come with **option argument**
 - `sort -k 1 data.txt`
 - // use the first column of data.txt as the key to sort

The most important command !!!



```
zhang@storm:~  
LS(1) User Commands LS(1)  
  
NAME  
    ls - list directory contents  
  
SYNOPSIS  
    ls [OPTION]... [FILE]...  
  
DESCRIPTION  
    List information about the FILES (the current directory by default).  
    Sort entries alphabetically if none of -cftuvSUX nor --sort.  
  
    Mandatory arguments to long options are mandatory for short options  
    too.  
  
    -a, --all  
        do not ignore entries starting with .  
  
    -A, --almost-all  
        do not list implied . and ..  
  
    --author  
        with -l, print the author of each file  
:  
:
```

man ls

- man: displaying online manuals

- Press **q** to quit, **space** to scroll down, **arrow** keys to roll up/down

Correcting type mistakes

- Shell starts to parse command line only when **Enter** key is pressed
- Delete the whole line (line-kill): C-u
- Erase a character: C-h or backspace key
- Many more fancy functionalities:
 - Auto-completion: press **Tab** key to ask shell to auto-complete command, or path name
 - History (repeat command): use arrow (up and down) keys to navigate past commands
 - ...

Shell: batch/scripting mode

- ◆ In batch mode, shell can interpret and execute shell scripts

```
#!/bin/bash
```

```
# count number of files/directories in curr. directory
```

```
ls -l | wc -l
```

- ◆ Shell constructs:

- variables,
- Loop and conditional statements
- I/O commands (read from keyboard, write to terminal)
- Function, arrays ...

Unix File

◆ Files: store information

- a sequence of 0 or more bytes containing arbitrary information

◆ What's in a filename?

- Case matters, no longer limited to 14 chars
- Special characters such as -, spaces are allowed, but you shouldn't use them in filename
 - ◆ Can you think of the reason ?
- Dot files are hidden, i.e., normally not listed by command `ls`
 - ◆ To display all files, including hidden files, use `ls -a`

What's in a file ?

◆ So far, we learnt that files are organized in a hierarchical directory structure

- Each file has a name, resides under a directory, is associated with some admin info (permission, owner)

◆ Contents of file:

- Text (ASCII) file (such as your C/C++ source code)
- Executable file (commands)
- A link to other files, ...
- Virtual file:
 - ◆ /proc: a pseudo-filesystem, contains user-accessible objects on runtime state of kernel and executing processes

◆ To check the type of file: "file <filename>"

File Viewing Commands

- cat: concatenate files and display on standard output (i.e., the terminal window)
 - cat [option] ... [file] ...
 - cat proj1.cpp
 - cat proj1.cpp proj2.cpp
 - cat -n proj1.cpp // display the file with line #
- More: file perusal filter (i.e., displaying file one screen at a time)
 - more proj1.cpp
- head, tail: display the beginning or ending lines of a file
 - tail -f output // display the file, append more lines as the file grows

*[] means the argument is optional
... means there can be multiple arguments of this type*

File manipulation commands

- rm: remove one or multiple files or directories
 - rm [option] ... FILE ...
 - rm temp
 - rm temp1 temp2
- Wildcards (metacharacter) can be used in command line
 - Letter * matches with any string
 - rm *.o: remove all .o files
 - ?: match any one character
 - [abc]: match with letter a or b or c
- rm -r: remove directories and their sub-dirs recursively
- rm -i : confirm with user before removing files

File manipulation commands (2)

◆ cp: copy file or directory

- cp [OPTION] SOURCE DESTINATION

◆ To make a backup copy of your program before dramatic change

- cp proj1.cpp proj1.cpp.bak

◆ To make a backup copy of a whole directory

- cp -r lab1_dir lab1_dir_backup

- -R, -r, --recursive: copy directories recursively

File manipulation commands (3)

◆ mv: move (rename) files/directories

- mv [OPTION] SOURCE DEST

- ◆ Rename SOURCE to DEST
- ◆ mv proj1.cpp lab1.cpp

- mv [OPTION]... SOURCE... DIRECTORY

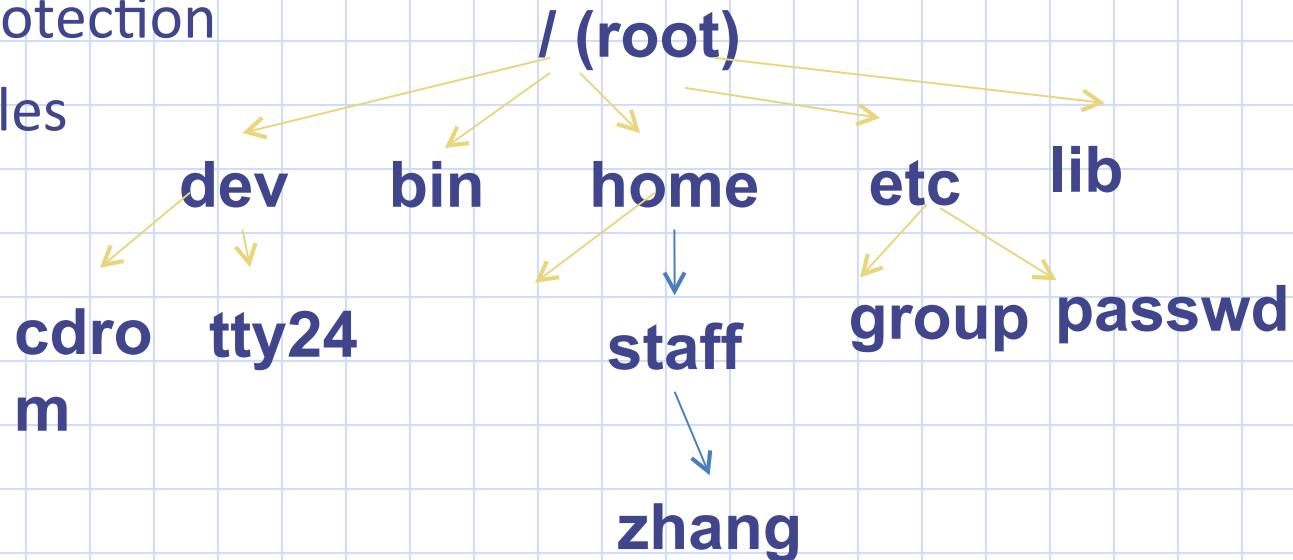
- ◆ Move SOURCE to DIRECTORY
- ◆ mv lab1.cpp lab2.cpp CISC3130

Hierarchical file system

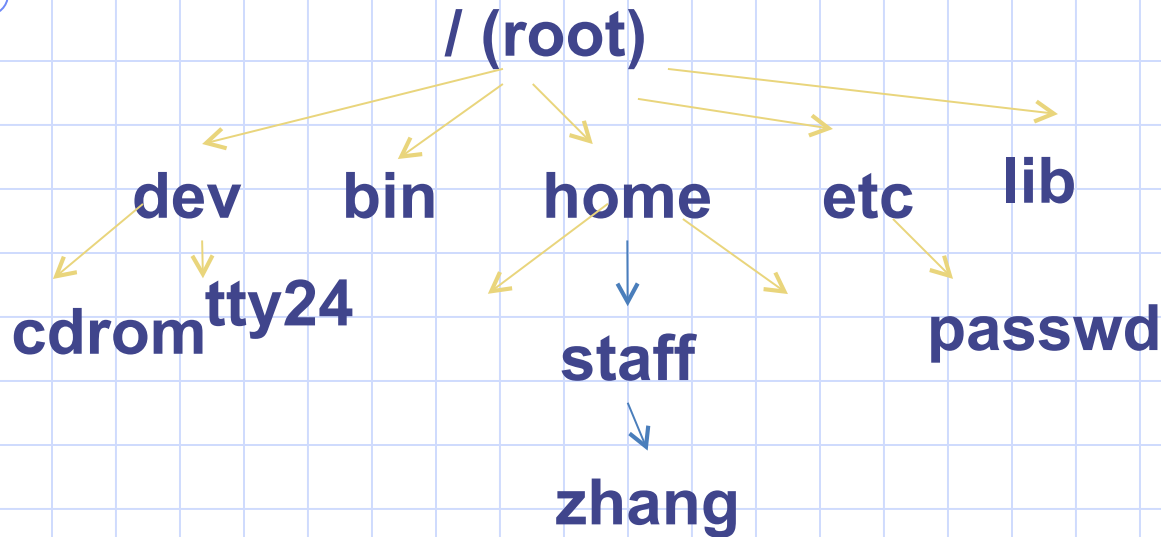
- Directory: a file that can hold other files

◆ Advantages of hierarchical file system:

- Files can have same names, as long as they are under different directories
- Easier for protection
- Organized files



Absolute pathname, path



- **Pathname** of a file/directory: location of file/directory in the file system
 - How do you tell other where your prog. is located ?
- **Absolute pathname**: path name specified relative to root, i.e., starting with the root (/)
 - e.g., /home/staff/zhang
 - What's the absolute pathname for the "passwd" file?

Home directory

- ◆ Every user has a **home directory** created for him/her
 - When you log in, you are in your home directory
 - In home directory, a user usually has permission to create files/directories, remove files ..
 - `~` to refer to current user's home directory
 - `~username` to refer to username's home directory

Current directory & Relative Pathname

- ◆ Tiring to specify **absolute pathname** each time
- ◆ To make life easier: working directory
 - User can move around the file system, shell remembers where the user is (i.e., current directory)
- ◆ To check your current directory, use command:
 - `pwd`

Getting around in the file system

- To create a subdirectory:
 - mkdir [option]... directory...
 - cd
 - mkdir CISC3130
 - cd CISC3130
 - mkdir lab1
- To remove a directory:
 - rmdir [option]... directory...
 - Report failure if directory is not empty
 - Can use rm -rf to remove non-empty directory

Command for change current directory (move around)

- cd [directory]

```
[zhang@storm Work]$ cd
```

```
[zhang@storm ~]$ pwd
```

```
/home/staff/zhang
```

```
[zhang@storm ~]$ cd Work
```

```
[zhang@storm Work]$ pwd
```

```
/home/staff/zhang/Work
```

```
[zhang@storm Work]$ cd ..
```

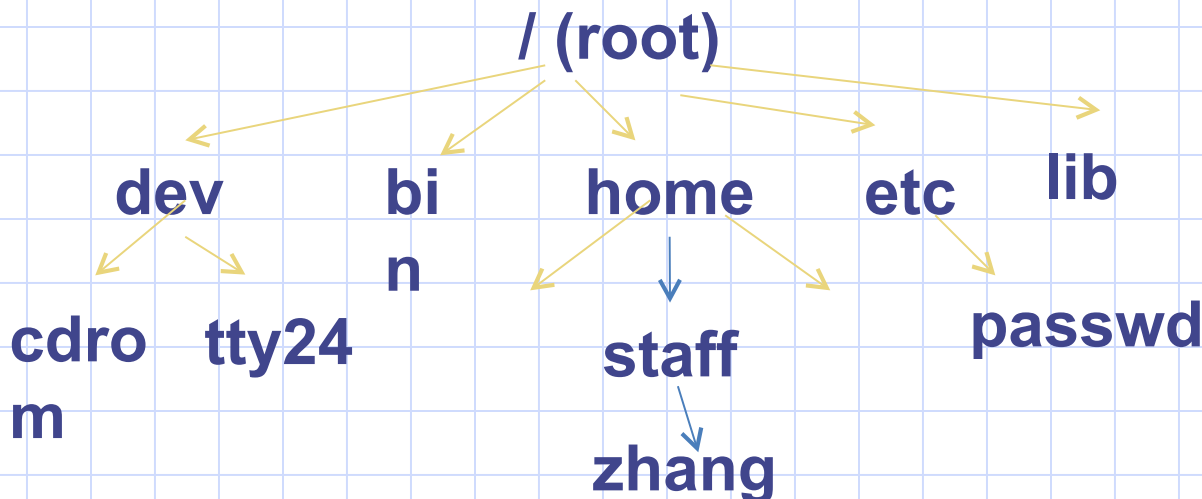
```
[zhang@storm ~]$ pwd
```

```
/home/staff/zhang
```

```
[zhang@storm ~]$
```

Relative pathname

- ◆ **Absolute pathname:** specified relative to root
- ◆ **Relative pathname:** specified relative to current directory
 - (current directory), .. (parent directory, one level up)
 - If current directory is at /home/staff/zhang, what is the relative pathname of the file passwd?
 - ◆ ../../../etc/passwd: go one level up, go one level up, go one level up, go to etc, passwd is there



Relative pathname

◆ For all commands that take file/directory name as arguments, you can use pathnames of the file/directory

◆ Example:

- `cd /home/staff/zhang/public_html`
- `pico CISC3130/index.html`
- `cd ..` (go up one level to parent directory)
- `cp ../prog2.cpp prog2.cpp`

Getting around in the file system

◆ ls: list directory contents

- ls [OPTION] ... [FILE]

ls: list files/directories under current directory

ls -l: long listing,

```
[zhang@storm CISC1600]$ ls -l
```

```
total 59180
```

```
-rw-r--r-- 1 zhang staff 509952 Sep  7 13:02 3_types.ppt
-rw-r--r-- 1 zhang staff 593408 Sep 14 23:38 4_computation.ppt
-rw-r--r-- 1 zhang staff 1297 Sep  2 12:18 account.html
-rw-r--r-- 1 zhang staff 3304448 Nov  7 18:24 ArrayVector1.ppt
drwxr-xr-x 2 zhang staff 4096 Dec  8 22:36 Codes
```

Long listing

◆ To get more information about each file

[zhang@storm Demo]\$ ls -al **Total disc space taken in blocks**

total 32

(1024 Byte)

drwxr-xr-x 5 zhang staff 4096 2008-01-16 16:01 .

drwxr-xr-x 41 zhang staff 4096 2008-01-16 16:01 ..

drwxr-xr-x 2 zhang staff 4096 2008-01-16 15:55 CCodes

-rw-r--r-- 1 zhang staff 38 2008-01-16 16:01 .HiddenFile

-rw-r--r-- 1 zhang staff 53 2008-01-16 15:57 README

drwxr-xr-x 2 zhang staff 4096 2008-01-16 15:55 SampleCodes

drwxr-xr-x 4 zhang staff 4096 2008-01-16 15:56

ShellScripts

User name of the owner and its group

Who has permission to read/write the file
directory

Long listing explained

d**rwx**r-x 4 zhang staff 4096 2008-01-16 15:56 ShellScriptes

◆ field 1

- 1st Character: specifies the type of the file.
 - ◆ - normal file, d directory, s socket file, l link file
- next 9 characters – File Permissions

◆ field 2: specifies the number of links for that file

◆ field 3 : specifies owner of the file

◆ field 4: specifies the group of the file

◆ field 5 : specifies the size of file.

◆ field 6: date and time of last modification of the file

◆ field 7: File name

File permissions

drwxr-xr-x 4 zhang staff 4096 2008-01-16 15:56 ShellScripts

◆ Each file is associated with permission info.

- Differentiate three type of users: owner user, users from same group as owner, others
- Three type of access: - in the field means no permission
 - ◆ Read (r): use "cat" to open a file to read, use "ls" to list files/directories under a directory
 - ◆ Write (w): modify the contents of the file, create/remove files from the directory
 - ◆ Execute (x): execute the file, or "cd" or "search" the directory for file

◆ Trying to list other's directory

```
[zhang@storm ~]$ ls ../roche/
```

```
ls: cannot open directory ../roche/: Permission denied
```

User and Group

- ◆ Each user has unique login name (user name), and corresponding numeric ID
 - Group id, home directory, login shell:
 - Stored in file `/etc/passwd`
- ◆ Groups: each user can belong to multiple groups
 - For collaboration ...
 - Group name, group id, user list info stored in `/etc/group`
- ◆ Superuser
 - ID: 0, username: root
 - Bypass all permission checks ...

More to play with

- ◆ who: show who is logged on
 - who am I
- ◆ write: write to another user's terminal (IMS?)
- ◆ which: show the full path name of a command
 - \$ which bash
 - /bin/bash
- ◆ How does shell find a command ?
 - Environment variable PATH stores a list of paths to search for programs: "set | grep PATH" or "echo \$PATH", "set" to show all variable settings
 - ◆ PATH=\$PATH:\$HOME/bin:.
 - Built-in commands: history, set, echo, etc.
- ◆ mail: send email from command line
 - mail -s "graded project" zhang < proj1.cpp

Summary

- ◆ Unix, a time-sharing operating system
- ◆ Operating system
 - a program that sits between hardware and user
 - Manages resources (CPU, memory, disk, network connection) and present user interface to user
- ◆ Unix programming environment, i.e., platform
 - the layered view

Summary (cont'd)

- PuTTY emulates a terminal
- Shell: a command line interface
 - Engage in dialog (a **session** with user)
- What's in a command
 - Name, and arguments
- Hierarchical file system: file and directory
- Commands for working with file system

References

1. Daniel P. Bovet and Marco Cesati, *Understanding the Linux Kernel*, 3rd edition, O'Reilly, 2005.
2. Robert Love, *Linux Kernel Development*, 2nd edition, Prentice-Hall, 2005.
3. Kwan Lowe, *Kernel Rebuild Guide*, <http://www.digitalhermit.com/linux/Kernel-Build-HOWTO.html>, 2004.
4. LWN, "Another Look at the New Development Model," <http://lwn.net/Articles/94605/>, 2004.
5. Joseph Pranovitch, *Wonderful World of Linux 2.2*, <http://kniggit.net/wwol22.html>, 1999.
6. Joseph Pranovitch, *Wonderful World of Linux 2.4*, <http://kniggit.net/wwol24.html>, 2001.
7. Joseph Pranovitch, *Wonderful World of Linux 2.6*, <http://kniggit.net/wwol24.html>, 2003.
8. Claudia Rodriguez et al, *The Linux Kernel Primer*, Prentice-Hall, 2005.
9. Andrew S. Tanenbaum, *Modern Operating Systems*, 2nd edition, Prentice-Hall, 2001.
10. Linus Torvalds and David Diamond, *Just For Fun: The Story of an Accidental Revolutionary*, Collins, 2001.